

Universidad Católica del Norte
Facultad de Ciencias Empresariales
Ingeniería Comercial — Sistemas de Información



Mati on the Way!

Informe Ejecutivo — Sistema de Gestión de Delivery de Almuerzos

Caso de Estudio	Caso 06 — Rutas Enredadas
Asignatura	Sistemas de Información
Profesores	Boris Bugueño — Alejandro Paolini
Fecha de Entrega	19 de junio de 2026
URL Producción	matiasfood.onrender.com
Repositorio	github.com/agustinargaluz/matiasfood

1. Resumen Ejecutivo

Mati on the Way! es una aplicación web de gestión de delivery de almuerzos desarrollada como solución tecnológica al Caso 06 "Rutas Enredadas". La aplicación resuelve los principales dolores operacionales de un servicio de delivery personal: la desorganización de pedidos, la falta de visibilidad en tiempo real de las rutas de entrega, la ausencia de análisis financiero y la inexistencia de un mecanismo de fidelización de clientes.

La solución implementada es un sistema web full-stack con tres módulos diferenciados: un portal de pedidos para clientes, una aplicación de ruta para el repartidor Matías y un panel de control analítico con exportación de reportes. La aplicación se encuentra en producción en matiasfood.onrender.com, accesible desde cualquier dispositivo móvil o de escritorio.

2. Análisis del Problema (Pain Points)

2.1 Contexto del Caso

El Caso 06 "Rutas Enredadas" describe la problemática de Matías, un estudiante universitario de 22 años que reparte almuerzos caseros en bicicleta a oficinas de la ciudad. Su negocio maneja entre 30 y 50 pedidos diarios, pero su método de organización es totalmente rudimentario: utiliza papelitos pegados en su pared para planificar las rutas cada mañana. Esta falta de herramientas digitales le impide escalar su negocio, generándole la sensación de que no puede crecer más.

El análisis del caso fue realizado mediante NotebookLM, cargando el documento de casos de estudio del curso como fuente principal. A partir de este análisis se identificaron los siguientes pain points y requerimientos del sistema.

2.2 Pain Points Identificados (NotebookLM)

#	Pain Point	Descripción (extraída del caso)	Impacto
1	Ineficiencia logística	Al organizar las rutas a mano, Matías termina recorriendo la misma zona dos veces en un mismo día, aumentando el tiempo y costo operacional.	Alto
2	Pérdida de información	Pierde pedidos porque los anota de forma incorrecta o descuidada en papelitos pegados en su pared, sin ningún respaldo digital.	Alto
3	Rigidez operativa	Cualquier solicitud de cambio en la dirección de entrega por parte de un cliente se convierte en un caos difícil de gestionar, sin mecanismo para actualizar la ruta en tiempo real.	Alto

4	Limitación del crecimiento	La carga administrativa manual consume su tiempo y capacidad operativa, bloqueando cualquier intento de expansión más allá de los 50 pedidos diarios actuales.	Alto
5	Sin analítica de negocio	No existe registro de ventas, zonas más rentables ni horas de mayor demanda que permita a Matías tomar decisiones informadas sobre su emprendimiento.	Medio
6	Sin fidelización de clientes	No existe mecanismo para incentivar la recompra ni premiar a los clientes frecuentes, limitando la retención y el crecimiento orgánico del negocio.	Medio

3. Especificación de Requerimientos

3.1 Requerimientos Funcionales

Los siguientes requerimientos fueron extraídos del análisis del Caso 06 mediante NotebookLM y luego implementados en la aplicación:

ID	Módulo	Requerimiento	Estado
R F0 1	Portal Cliente	Registro de pedidos con nombre, teléfono, dirección y selección de productos del menú.	✓ Implementado
R F0 2	Portal Cliente	Menú del día rotativo con foto diferente por día de la semana (7 variaciones).	✓ Implementado
R F0 3	Portal Cliente	Selección de múltiples productos y bebidas con control de cantidad por ítem.	✓ Implementado
R F0 4	Matipuntos	Sistema de puntos de fidelización: acumulación automática (1 punto por \$1.000) y canje por productos.	✓ Implementado
R F0 5	App Repartidor	Visualización de pedidos activos agrupados por zona geográfica con mapa interactivo Leaflet/OpenStreetMap.	✓ Implementado
R F0 6	App Repartidor	Geocodificación automática de direcciones mediante API Nominatim con caché local.	✓ Implementado
R F0 7	App Repartidor	GPS en tiempo real: marcador 🚲 que sigue la posición de Matías mientras pedalea.	✓ Implementado
R F0	App Repartidor	Notificaciones automáticas de pedidos nuevos: banner, vibración y notificación del sistema.	✓ Implementado

8			
R F0 9	App Repartidor	Marcado de entregas con actualización en tiempo real y archivado automático de entregados.	✓ Implementado
R F1 0	Dashboard	KPIs del día: total pedidos, entregados, pendientes, ventas y eficiencia.	✓ Implementado
R F1 1	Dashboard	Calendario analítico mensual con selección de día para ver detalle y resumen del mes.	✓ Implementado
R F1 2	Dashboard	Exportación a Excel con formato de tabla: reporte diario y mensual con costos, ventas y ganancia neta.	✓ Implementado
R F1 3	Dashboard	Historial de clientes ordenado por frecuencia de pedidos con teléfono clickeable.	✓ Implementado
R F1 4	Backend	API REST con endpoints: POST /pedidos, GET /pedidos/hoy, GET /pedidos/historial, GET /dashboard/hoy, GET /clientes.	✓ Implementado

3.2 Requerimientos No Funcionales

ID	Atributo	Descripción
R N F0 1	Disponibilidad	La aplicación debe estar disponible 24/7. Se logra mediante deploy en Render (backend) y GitHub Pages (frontend) con uptime garantizado.
R N F0 2	Responsividad	La interfaz debe funcionar correctamente en dispositivos móviles (el repartidor opera desde su celular). Diseño mobile-first con max-width 420px.
R N F0 3	Rendimiento	Tiempo de respuesta del backend menor a 2 segundos. Las imágenes se sirven desde GitHub CDN. El dashboard se refresca automáticamente cada 30 segundos.
R N F0 4	Seguridad	CORS habilitado en el backend para permitir peticiones cross-origin desde GitHub Pages. Datos de clientes almacenados solo en la base de datos del servidor.
R N F0 5	Mantenibilidad	Código organizado en 3 archivos HTML independientes + 1 backend Python. Repositorio público en GitHub con historial de commits.
R N	Usabilidad	Interfaz intuitiva sin manual de usuario. Flujo de pedido completable en menos de 60 segundos. Feedback visual inmediato en cada

F0 6		acción.
---------	--	---------

4. Arquitectura y Stack Tecnológico

4.1 Stack Tecnológico y Justificación

Capa	Tecnología	Justificación
Frontend	HTML5 + CSS3 + JavaScript (Vanilla)	Sin dependencias de compilación. Deploy instantáneo en GitHub Pages. Carga ultrarrápida en dispositivos móviles de baja potencia. Ideal para una app usada en campo.
Mapas	Leaflet.js + OpenStreetMap	Open-source, sin costo por requests ni API key requerida. Geocodificación via Nominatim. Soporte nativo para marcadores personalizados y rutas.
Backend	Python 3.11 + FastAPI	Framework moderno con documentación automática (OpenAPI). Tipado fuerte con Pydantic. Rápido desarrollo de APIs REST. Amplia compatibilidad con Render.
Base de Datos	SQLite 3	Serverless, sin configuración adicional. Perfecta para un negocio unipersonal con carga media-baja. El archivo .db se inicializa automáticamente en el primer arranque.
Deploy Backend	Render.com (Web Service)	Deploy automático desde GitHub. Free tier suficiente para el volumen del proyecto. HTTPS incluido. URL pública estable.
Deploy Frontend	GitHub Pages	CDN global gratuito. Deploy automático al hacer push. Sin configuración de servidor. Integración nativa con el repositorio.
IA Generativa	Claude (Anthropic) — claude.ai	Asistente principal de pair-programming utilizado en todas las fases de desarrollo, depuración, refactorización y documentación del proyecto.

4.2 Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas clásica:

Capa de Presentación	Capa de Negocio	Capa de Datos
 index.html Portal de pedidos para clientes	 main.py (FastAPI)	 SQLite (ruta.db)
 matias.html	Endpoints REST: POST /pedidos	Tablas: • zonas

App del repartidor con mapa GPS



dashboard.html

Panel analítico y exportación

```
GET /pedidos/hoy
GET /pedidos/historial
PATCH
/pedidos/{id}/entregar
GET /dashboard/hoy
GET /clientes
```

- clientes
- menus
- pedidos

*Auto-inicializada con datos
semilla en primer arranque.*

5. Solución Implementada

5.1 Módulo 1: Portal de Pedidos (index.html)

El portal de pedidos es la interfaz que utilizan los clientes para realizar sus pedidos. Está diseñado con un enfoque mobile-first y permite completar un pedido en menos de 60 segundos.

- Menú del día rotativo con 7 variaciones (una por día de la semana), con foto y descripción actualizadas automáticamente.
- Catálogo completo: hamburguesas clásica, BBQ y vegana; completos italiano y vegano; 5 bebidas con foto real.
- Control de cantidad individual por producto mediante botones +/-.
- Resumen dinámico en tiempo real con cálculo automático de totales y Matipuntos a ganar.
- Sistema Matipuntos: acumulación de 1 punto por cada \$1.000 pedido, canje por productos desde la misma app.
- Perfil de cliente con foto personalizable (almacenado en localStorage).

5.2 Módulo 2: App del Repartidor (matias.html)

La aplicación del repartidor es el corazón operacional del sistema. Está optimizada para uso en terreno desde un smartphone.

- Mapa interactivo con todos los pedidos del día, marcados con íconos numerados por orden de entrega y colores por zona geográfica.
- Geocodificación automática de direcciones mediante API Nominatim (OpenStreetMap), con caché local para evitar peticiones repetidas.
- GPS en tiempo real: activable con un botón, muestra el marcador 🚲 con la posición exacta de Matías sobre el mapa.
- Notificaciones de pedidos nuevos: polling cada 30 segundos, banner de aviso, vibración del teléfono y notificación nativa del sistema operativo.
- Botón de llamada directa al cliente desde cada tarjeta de pedido.
- Integración con Google Maps para navegación en modo bicicleta.
- Marcado de entrega con actualización instantánea del mapa y archivado automático.

5.3 Módulo 3: Dashboard Analítico (dashboard.html)

El panel de control permite a Matías analizar el desempeño de su negocio con datos reales del backend, actualizados automáticamente cada 30 segundos.

- KPIs en tiempo real: pedidos totales, entregados, pendientes, ventas del día y eficiencia (%).
- Lista de pedidos del día sincronizada con el backend (sin datos demo hardcodeados).

- Gráfico de zonas con distribución de pedidos por sector geográfico.
- Calendario analítico mensual: visualización día a día con indicadores de actividad y resumen mensual.
- Resumen mensual: total pedidos, total ventas, eficiencia promedio y mejor día del mes.
- Historial de clientes: tabla ordenada por frecuencia de compra con teléfono clickeable.
- Exportación a Excel con formato de tabla profesional: reporte diario (Nº, cliente, pedido, precio, costo estimado, ganancia neta, estado) y reporte mensual con resumen financiero (ventas, costos, ganancia neta, margen %).

5.4 Base de Datos

La base de datos SQLite se inicializa automáticamente en el primer arranque del servidor con estructura y datos semilla. El esquema relacional implementado es:

Tabla	Campos Principales	Descripción
zonas	id, nombre, descripcion, color_hex	3 zonas de reparto: Centro La Serena, Coquimbo, Las Compañías.
clientes	id, nombre_completo, telefono, direccion_defecto, zona_id, creado_en	Registro persistente de clientes. Se crea o actualiza automáticamente al recibir un pedido.
menus	id, nombre, descripcion, precio, disponible	Catálogo de productos: menú del día, hamburguesas, completos.
pedidos	id, cliente_id, menu_id, zona_id, direccion_entrega, notas, estado, orden_ruta, fecha_pedido, fecha_entrega	Tabla principal. El campo estado fluye de pendiente → en_ruta → entregado. orden_ruta define el orden óptimo de entrega.

6. Estrategia de Prompt Engineering

El desarrollo completo de la aplicación fue realizado con asistencia de Claude (Anthropic) mediante claude.ai. A continuación se describen las estrategias de prompting más efectivas utilizadas durante el proyecto.

6.1 Estrategias Aplicadas

Estrategia	Descripción y Ejemplo
Contexto Completo	Se proporcionó siempre el contexto completo del proyecto: nombre de la app, stack tecnológico, URL de producción y problema a resolver. Ejemplo: "Somos Mati on the Way!, app delivery en FastAPI + HTML. El backend está en matiasfood.onrender.com. Necesito que..."
Especificidad del Error	En lugar de "no funciona", se describió el comportamiento exacto con capturas. Ejemplo: "El Excel exportado muestra todo en la columna A porque usa \t como separador, pero Excel Chile usa ;".
Iteración Incremental	Las funcionalidades complejas se construyeron en pasos: primero estructura básica, luego estilo, luego integración con backend, luego corrección de bugs.
Validación Explícita	Se solicitó verificación automática del código mediante scripts Python que chequeaban la presencia de elementos clave. Esto permitió detectar funciones duplicadas, datos demo no eliminados y bugs de lógica.
Evidencia Visual	Se enviaron capturas de pantalla de los problemas visuales (Excel mal formateado, fotos incorrectas, datos demo visibles). Esto permitió al asistente identificar y corregir problemas que no eran evidentes solo con el código.
Restricción de Scope	Cuando la IA entregaba más de lo pedido (o menos), se especificó exactamente qué archivos modificar. Ejemplo: "solo modifica el dashboard.html, no toques main.py".

6.2 Lecciones Aprendidas del Proceso

- La verificación automática de código (scripts de check) fue la estrategia más efectiva para garantizar calidad: detectó funciones duplicadas, referencias a datos demo y bugs de lógica que el ojo humano no habría notado fácilmente.
- Las capturas de pantalla fueron esenciales para problemas visuales: el problema de las fotos incorrectas (hot dog = brócoli emoji, sprite = Pepsi) solo fue resoluble una vez que se vio la evidencia visual.
- La iteración es parte del proceso: el Excel tardó 3 iteraciones hasta quedar correcto (tab → punto y coma → formato HTML table con estilos inline).

- La especificidad del error acelera la solución: "todo en columna A" es mejor que "el Excel no funciona".

7. Conclusiones

El proyecto *Mati on the Way!* demuestra cómo un emprendimiento de delivery unipersonal puede transformar su operación mediante una solución tecnológica desarrollada con asistencia de Inteligencia Artificial Generativa. Los resultados obtenidos permiten establecer las siguientes conclusiones:

- La aplicación resuelve íntegramente los 5 pain points identificados en el Caso 06: la gestión de pedidos es ahora digital y persistente; la ruta está visualizada en un mapa con GPS real; existen analíticas financieras detalladas; hay un sistema de fidelización (Matipuntos); y las notificaciones automáticas eliminan la coordinación reactiva.
- La arquitectura seleccionada (HTML vanilla + Python/FastAPI + SQLite) fue apropiada para el contexto del proyecto: bajo costo operacional, deploy sencillo, sin dependencias complejas y rendimiento suficiente para el volumen esperado.
- El uso de IA generativa como asistente de pair-programming aceleró significativamente el desarrollo, permitiendo implementar en pocas sesiones funcionalidades que habrían tomado semanas de desarrollo tradicional.
- El proceso iterativo de desarrollo con Claude evidenció que la calidad de los prompts es directamente proporcional a la calidad del resultado: cuanto más específico y contextualizado el prompt, menos iteraciones fueron necesarias.
- La aplicación está en producción y es funcional: puede recibir pedidos reales, el repartidor puede gestionar su ruta desde el celular y el análisis financiero es exportable a Excel para contabilidad básica del negocio.

8. URLs de Acceso al Sistema

Recurso	URL
Portal cliente (producción)	https://matiasfood.onrender.com/
App repartidor	https://matiasfood.onrender.com/matias
Dashboard analítico	https://matiasfood.onrender.com/dashboard
Frontend alternativo (GitHub Pages)	https://agustinargaluz.github.io/matiasfood/
Repositorio GitHub	https://github.com/agustinargaluz/matiasfood
API REST (docs automáticos)	https://matiasfood.onrender.com/docs